ACTIVITY - 6

The 8-Puzzle Problem

Detail	Information
Name	D Jeshurun Silas
Registration No.	192525324
Course	Artificial Intelligence
Course Code	CSA1718
Activity	8-Puzzle: Initial State to Goal State

1. Introduction

Artificial Intelligence (AI) is the branch of computer science that tries to make machines solve problems in a way that looks intelligent. One of the most common ways AI solves a problem is by searching: the computer tries different possible actions one after another until it finds a sequence of actions that reaches the desired result.

To learn and test such search methods, AI students use small, well-defined problems called "toy problems". The 8-puzzle is one of the most famous of them. It is simple enough to understand in a minute, yet rich enough to show how real AI search algorithms work. If you have ever played a sliding-tile puzzle (the kind where you slide squares around inside a small frame to make a picture or a number order), you have already played a version of the 8-puzzle.

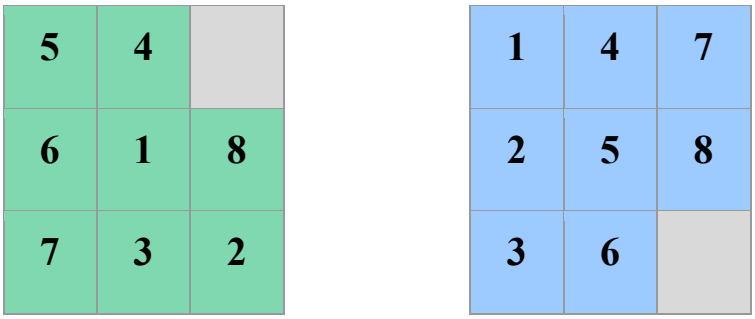
This report explains the problem from the very basics, shows how it is converted into an AI problem, describes the algorithms used to solve it, and then applies the ideas to the specific puzzle given in this activity. Along the way, it reveals an important finding: the given puzzle cannot actually be solved, and the report explains exactly why.

2. Problem Statement

The 8-puzzle consists of a 3 × 3 square board (3 rows and 3 columns, so 9 positions) and 8 square tiles numbered 1 to 8. Because there are 9 positions but only 8 tiles, exactly one position is always empty. This empty position is called the blank.

A tile that is directly next to the blank (above, below, to the left or to the right of it) can be slid into the blank. When a tile slides into the blank, the blank takes the place the tile just left. The objective is to start from a given starting arrangement (the Initial State) and, by sliding tiles one move at a time, reach a required final arrangement (the Goal State).

The two arrangements given in this activity are shown below. The grey square is the blank.



Initial State **Goal State**

Reading the initial state row by row: Row 1 is 5, 4, blank. Row 2 is 6, 1, 8. Row 3 is 7, 3, 2.

Reading the goal state row by row: Row 1 is 1, 4, 7. Row 2 is 2, 5, 8. Row 3 is 3, 6, blank.

Two ways to think about a move. We can say "a tile slides into the blank", or equivalently, "the blank moves up, down, left or right". Both describe exactly the same change on the board. In AI, the second view is more convenient because there is only one blank, so there are only four possible actions: Up, Down, Left and Right.

3. Key Terms Explained in Simple Words

Before going deeper, the table below explains the technical words used in this report.

Term	Simple meaning
State	One complete arrangement of the board at a given moment. Every different arrangement of the tiles is a different state.
Initial state	The arrangement we start from.
Goal state	The arrangement we want to reach.

Term	Simple meaning
Action / Operator	A legal move: moving the blank Up, Down, Left or Right.
State space	The collection of all states that can possibly be reached from the initial state by making moves. Think of it as a huge map in which every location is a board arrangement.
Search	The process of exploring the state space, move by move, to find a path from the initial state to the goal state.
Path cost	The price of a solution. Here each move costs 1, so the cost equals the number of moves.
Optimal solution	A solution that uses the smallest possible number of moves.
Heuristic	An educated guess of how far a state is from the goal. It helps the search choose promising moves first.
Admissible heuristic	A heuristic that never over-estimates the true distance to the goal. Such a heuristic is "safe" and keeps A* search optimal.
Inversion	A pair of tiles that are in the "wrong order" with respect to each other. This is explained fully in Section 8.
Parity	Whether a number is even or odd.

4. How the Moves Work

The number of moves available depends on where the blank is. The blank cannot leave the board, so a blank near the edge has fewer choices than a blank in the middle.

Blank position	Number of moves	Explanation
Corner (4 such positions)	2	Two of its four sides touch the border of the board.
Edge, not corner (4 such positions)	3	One side touches the border.
Centre (1 position)	4	All four sides have a neighbour.

In the initial state of this activity, the blank is in the top-right corner. It can therefore only move Down (swapping with tile 8) or Left (swapping with tile 4). The two possible next states are shown below.

5	4	8
6	1	
7	3	2

After moving blank Down

5		4
6	1	8
7	3	2

After moving blank Left

Every move is also reversible: if you slide a tile into the blank, you can slide it straight back. This means that the state space has no "dead ends" in the sense of being unable to undo a move, and it also means that if state A can reach state B, then B can reach A.

5. Formulating the Puzzle as an AI Problem

To let a computer solve a problem, we first describe it in a precise, formal way. This is called problem formulation. For the 8-puzzle, the formulation has five parts:

Component	8-puzzle definition
States	Any placement of the tiles 1 to 8 and the blank on the 3 × 3 board.
Initial state	5 4 _ / 6 1 8 / 7 3 2 (as given).
Actions	Move the blank Up, Down, Left or Right, provided it stays on the board.
Transition model	The new state is the old state with the blank and the neighbouring tile swapped.
Goal test	Is the current state exactly 1 4 7 / 2 5 8 / 3 6 _ ?
Path cost	Each move costs 1; total cost = number of moves.

5.1 How big is the state space?

There are 9 positions and 9 "objects" (8 tiles plus the blank). The number of ways to arrange 9 different objects in 9 positions is 9! (nine factorial) = 9 × 8 × 7 × 6 × 5 × 4 × 3 × 2 × 1 = 362,880.

Important: only half of these arrangements can be reached from any chosen starting arrangement. That is, from a given initial state, exactly 181,440 states are reachable and the other 181,440 are completely out of reach, no matter how many moves are made. The puzzle therefore splits into two separate "worlds" that can never be connected. Section 8 explains why, and shows that our goal state lies in the other world.

5.2 The search tree

A search algorithm explores the state space as a tree. The initial state is the root. Each possible move creates a child state. Each child has its own children, and so on. A solution is a path from the root down to a node that matches the goal state. The tree can become very large, which is why smart search methods are needed.

6. Search Algorithms for the 8-Puzzle

A search algorithm is simply a rule that decides which state to explore next. Different rules give different behaviour in terms of speed, memory use and quality of the answer. The main ones are described below in plain language.

6.1 Uninformed (blind) search

These methods know nothing about the puzzle except its rules. They do not know whether a move is "getting closer" to the goal.

- **Breadth-First Search (BFS):** Explore all states that are 1 move away, then all states 2 moves away, then 3 moves away, and so on, level by level. Because it checks shorter paths first, the first solution it finds uses the fewest moves. Its weakness is memory: it must remember a very large number of states.
- **Depth-First Search (DFS):** Pick one move, then keep making moves in that direction as deep as possible, and only come back (backtrack) when stuck. It uses little memory but can wander for a very long time, and the solution it finds may be much longer than necessary.
- **Uniform Cost Search (UCS):** Always expand the state with the lowest total cost so far. In the 8-puzzle all moves cost 1, so UCS behaves exactly like BFS.

6.2 Informed (heuristic) search

These methods use extra knowledge, a heuristic, to estimate how close a state is to the goal. They try the most promising states first, which saves a lot of time.

- **Greedy Best-First Search:** Always expand the state that looks closest to the goal (smallest h). It is fast but may not give the shortest solution.
- **A* (A-star) Search:** Combines the cost already paid, g(n), with the estimated cost still to go, h(n): f(n) = g(n) + h(n). It always expands the state with the smallest f. If h never over-estimates the real distance (it is admissible), A* is guaranteed to find the shortest solution, and it is usually much faster than BFS.

6.3 Comparison

Algorithm	Finds a solution if one exists?	Shortest solution?	Remarks
BFS	Yes	Yes	Uses a lot of memory.
DFS	Not always (may loop)	No	Very low memory; needs a depth limit or repeated-state check.
UCS	Yes	Yes	Same as BFS when all costs are equal.
Greedy	Not always	No	Fast, but can be misled by the heuristic.
A*	Yes	Yes (with admissible h)	Best general choice for the 8-puzzle.

7. Heuristics: Guessing the Distance to the Goal

A good heuristic makes search faster. Two popular heuristics for the 8-puzzle are explained below and then applied to the initial state of this activity.

7.1 Heuristic h1: number of misplaced tiles

Count how many tiles (not counting the blank) are not in their goal position. Since each misplaced tile needs at least one move, the real number of moves can never be smaller than this count. So h1 is admissible.

7.2 Heuristic h2: Manhattan distance

For every tile, count how many rows and columns it must travel to reach its goal square, then add up these numbers for all tiles. The name comes from walking in a city like Manhattan, where you can only move along streets (horizontally or vertically), never diagonally. Because a tile can only move one square per move, h2 never over-estimates, and it is more accurate than h1.

7.3 Worked calculation for the initial state

Take each tile of the initial state and compare its current square with its square in the goal state. Rows are counted from the top (1 to 3) and columns from the left (1 to 3).

Tile	Current (row, col)	Goal (row, col)	Misplaced?	Manhattan distance
1	(2, 2)	(1, 1)	Yes	1 + 1 = 2
2	(3, 3)	(2, 1)	Yes	1 + 2 = 3
3	(3, 2)	(3, 1)	Yes	0 + 1 = 1

Tile	Current (row, col)	Goal (row, col)	Misplaced?	Manhattan distance
4	(1, 2)	(1, 2)	No	0
5	(1, 1)	(2, 2)	Yes	1 + 1 = 2
6	(2, 1)	(3, 2)	Yes	1 + 1 = 2
7	(3, 1)	(1, 3)	Yes	2 + 2 = 4
8	(2, 3)	(2, 3)	No	0
Total			h1 = 6	h2 = 14

So for the initial state, h1 = 6 and h2 = 14. This means that, if the puzzle were solvable, at least 14 moves would be required. Notice that these heuristics only estimate distance; they cannot tell us whether the goal is reachable at all. That question is answered in the next section.

8. Is the Puzzle Even Solvable?

Before spending time on any search, a clever AI programmer first checks whether a solution can exist. For the 8-puzzle there is a simple and very fast test based on inversions.

8.1 What is an inversion?

Write all the tiles in one line by reading the board row by row, from left to right and top to bottom, and ignore the blank. An inversion is a pair of tiles where a bigger number appears before a smaller number.

Tiny example: In the line 2, 1, 3 there is one inversion (the pair 2 and 1, because 2 comes before 1). In the line 3, 2, 1 there are three inversions (3 and 2, 3 and 1, 2 and 1). In the line 1, 2, 3 there are none.

8.2 The rule

> **Solvability rule for the 3 × 3 board**
>
> Two arrangements can be turned into each other if and only if their inversion counts are both even or both odd (that is, they have the same parity). If one count is even and the other is odd, it is impossible.

8.3 Why does this rule work?

Look at what a single move does to the line of tiles:

- **Moving the blank Left or Right** (a horizontal move): the tile that moves stays in the same row, so the order of the line of tiles does not change at all. The inversion count stays the same.
- **Moving the blank Up or Down** (a vertical move): the moving tile jumps over exactly two other tiles in the line. Each jump changes the order of that tile with each of the two tiles it passes. Each of those two pairs either gains one inversion or loses one, so the total change is +2, 0 or -2.

In both cases the inversion count changes by an even number (0, +2 or -2), so it can never switch from even to odd or from odd to even. The parity is permanently locked by the rules of the puzzle. This is why exactly half of the 362,880 arrangements are reachable from any given start.

8.4 Applying the rule to our puzzle

Initial state. Line of tiles: 5, 4, 6, 1, 8, 7, 3, 2.

Tile	Inversions with later tiles	Which later tiles are smaller?
5	4	4, 1, 3, 2
4	3	1, 3, 2
6	3	1, 3, 2
1	0	none
8	3	7, 3, 2
7	2	3, 2
3	1	2
2	0	none
Total	16	Even

Goal state. Line of tiles: 1, 4, 7, 2, 5, 8, 3, 6.

Tile	Inversions with later tiles	Which later tiles are smaller?
1	0	none
4	2	2, 3
7	4	2, 5, 3, 6

Tile	Inversions with later tiles	Which later tiles are smaller?
2	0	none
5	1	3
8	2	3, 6
3	0	none
6	0	none
Total	9	Odd

Result of the check

The initial state has 16 inversions (even) and the goal state has 9 inversions (odd). Their parities are different, so the goal state can NEVER be reached from the given initial state, however cleverly the tiles are moved.

9. Verification by Exhaustive Search

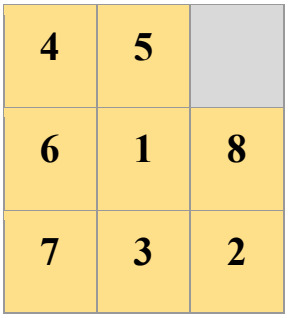
To make sure the mathematical rule was applied correctly, a breadth-first search program was run starting from the given initial state. BFS keeps exploring every new arrangement it can create, and remembers the ones it has already seen so it never repeats itself.

Item	Result
Total arrangements of the board (9!)	362,880
Distinct states reached from the initial state	181,440
Goal state among the reached states?	No

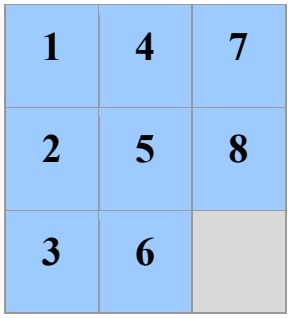
The search reached exactly 181,440 states, which is exactly half of all arrangements, as predicted by the theory, and the goal state was not one of them. This independently confirms the inversion-parity result.

10. What Can Be Done? A Solvable Version

Since the goal is unreachable, the problem cannot be completed as stated. The usual remedy is to correct the puzzle so that both states have the same parity. Swapping any two numbered tiles in one of the states flips its parity. As an illustration, we swap tiles 5 and 4 in the initial state:



Modified Initial State



Goal State

Now the line of tiles is 4, 5, 6, 1, 8, 7, 3, 2, which has 15 inversions (odd), matching the goal (9, odd). The puzzle is solvable. The Manhattan distance is still 14, so at least 14 moves are needed. Running BFS finds the shortest solution, which takes 24 moves. The moves below describe the movement of the blank, starting from the top-right corner.

Moves	1st	2nd	3rd	4th	5th	6th
1-6	Left	Down	Left	Down	Right	Up
7-12	Left	Up	Right	Down	Left	Down
13-18	Right	Right	Up	Up	Left	Down
19-24	Down	Left	Up	Right	Right	Down

Each row lists six consecutive moves, read from left to right, starting with move 1. After the 24th move the board matches the goal state exactly. The solution length (24) is larger than the heuristic lower bound (14), which is normal: the heuristic only gives a minimum, and real solutions usually need extra moves to get tiles around each other.

11. Why Does This Matter? Real-World Relevance

The 8-puzzle may look like a game, but the ideas behind it are used everywhere in AI and computing:

- Route planning: GPS navigation systems use A* search with a distance heuristic to find the shortest route between two places.

- Robotics: a robot arm planning a sequence of movements to reach a position is solving a state-space search problem.
- Game playing: chess and other game programs search through possible moves in the same way, using heuristics to evaluate positions.
- Scheduling and logistics: arranging tasks, delivery routes or timetables can be modelled as searching through many possible configurations.
- Feasibility checking: just like the parity test here, many real systems first check whether a problem is solvable before running an expensive search, saving a great deal of computing time.

12. Conclusion

The 8-puzzle was studied as a classic state-space search problem. It was formulated using states, actions, a transition model, a goal test and a path cost. The main search strategies, BFS, DFS, UCS, Greedy search and A*, were explained, and two heuristics (misplaced tiles and Manhattan distance) were calculated for the given initial state, giving values of 6 and 14 respectively.

The most important finding is that the puzzle given in this activity cannot be solved. The initial state has 16 inversions (even) while the goal state has 9 inversions (odd). Because every legal move preserves the parity of the inversion count, the goal state is in the unreachable half of the 362,880 possible arrangements. An exhaustive BFS over all 181,440 reachable states confirmed this.

When the puzzle was adjusted by swapping two tiles (5 and 4) in the initial state, it became solvable, and the shortest solution was found to be 24 moves. The activity demonstrates two key lessons of AI problem solving: represent the problem precisely, and check solvability before searching.

13. Summary of Results

Item	Value
Board size	3 × 3 (8 tiles + 1 blank)
Total possible states	362,880 (9!)
States reachable from the initial state	181,440
Inversions in initial state	16 (even)
Inversions in goal state	9 (odd)
Misplaced tiles (h1) for initial state	6
Manhattan distance (h2) for initial state	14

Item	Value
Is the given goal reachable?	No
Shortest solution after swapping tiles 5 and 4	24 moves